

# DevOps Intern Final Assessment

---

**Name:** Yajat Prabhakar

**Date:** August 18, 2026

## Project Description

This repository contains the DevOps pipeline for the assessment.

The pipeline covers:

Git repo → shell script → Docker image → CI pipeline →  
Nomad job → Loki logs

## 1. Git & GitHub Setup

The repository contains this README and `hello.py`.

`hello.py` prints:

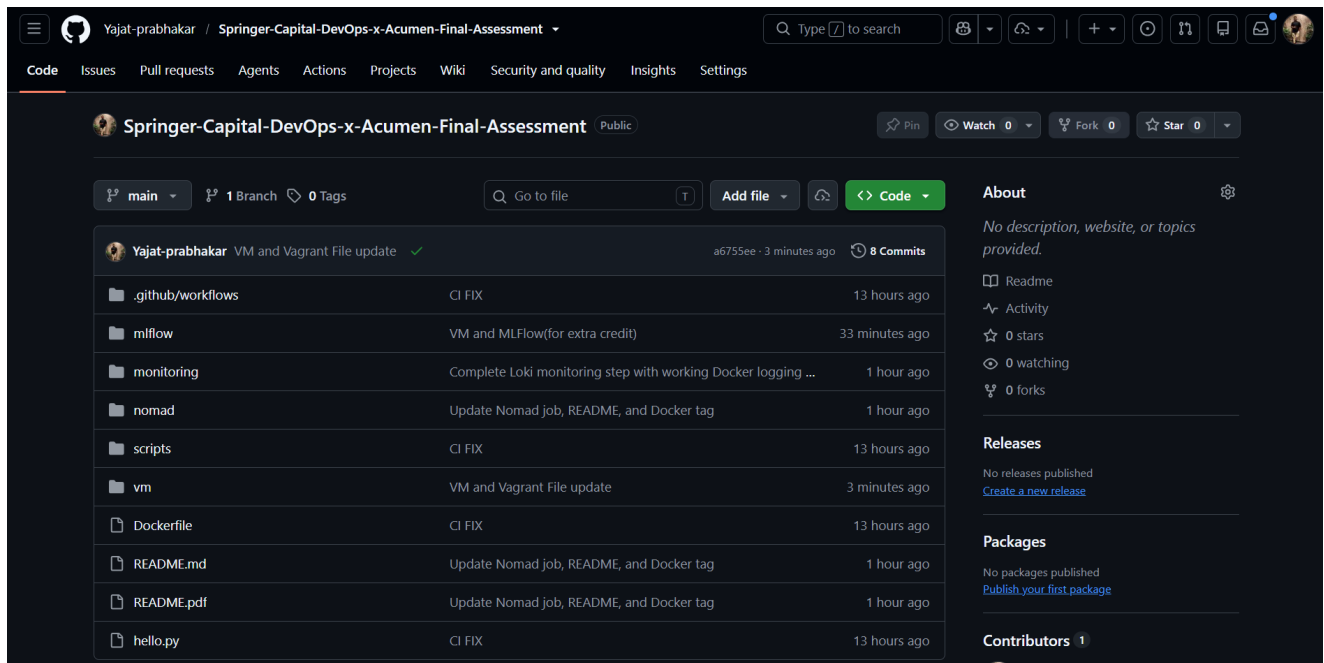
```
Hello, DevOps!
```

Run it directly:

```
python3 hello.py
```

Output:

```
Hello, DevOps!
```



## 2. Linux & Scripting Basics

`scripts/sysinfo.sh` prints the current user, date, and disk usage.

Make it executable:

```
chmod +x scripts/sysinfo.sh
./scripts/sysinfo.sh
```

**Actual output:**

```
yajat
Wed Aug 19 05:57:59 UTC 2026
Filesystem      Size  Used Avail Use% Mounted on
none            4.9G   0    4.9G   0% /usr/lib/modules/...
/dev/sdf        1007G  1.7G  954G   1% /
C:\             475G  378G   98G   80% /mnt/c
...
```

```
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assessment$ chmod +x scripts/sysinfo.sh
./scripts/sysinfo.sh
yajat
Wed Aug 19 05:57:59 UTC 2026
Filesystem                                Size  Used Avail Use% Mounted on
none                                     4.9G   0 4.9G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none                                     4.9G  4.0K 4.9G   1% /mnt/wsl
drivers                                 475G  378G  98G   80% /usr/lib/wsl/drivers
/dev/sdf                               1007G   1.7G 954G   1% /
none                                     4.9G  116K 4.9G   1% /mnt/wslg
none                                     4.9G   0 4.9G   0% /usr/lib/wsl/lib
rootfs                                  4.9G   2.7M 4.9G   1% /init
none                                     4.9G  560K 4.9G   1% /run
none                                     4.9G   0 4.9G   0% /run/lock
none                                     4.9G   0 4.9G   0% /run/shm
none                                     4.9G   76K 4.9G   1% /mnt/wslg/versions.txt
none                                     4.9G   76K 4.9G   1% /mnt/wslg/doc
C:\                                     475G  378G  98G   80% /mnt/c
tmpfs                                   4.9G   0 4.9G   0% /tmp
none                                     1.0M   0 1.0M   0% /run/credentials/getty@tty1.service
tmpfs                                   995M  12K 995M   1% /run/user/1000
none                                     1.0M   0 1.0M   0% /run/credentials/systemd-journald.service
none                                     1.0M   0 1.0M   0% /run/credentials/systemd-resolved.service
none                                     1.0M   0 1.0M   0% /run/credentials/console-getty.service
tmpfs                                   995M  12K 995M   1% /run/user/0
none                                     4.9G  724K 4.9G   1% /mnt/wsl/docker-desktop/shared-sockets/host-services
/dev/sdd                                136M   71M  55M   57% /mnt/wsl/docker-desktop/docker-desktop-user-distro
/dev/loop0                              812M  812M   0 100% /mnt/wsl/docker-desktop/cli-tools
C:\Program Files\Docker\Docker\resources 475G  378G  98G   80% /Docker/host
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assessment$ |
```

### 3. Docker Basics

The **Dockerfile** uses Python 3.12-slim as the base image and runs **hello.py**.

Build the image:

```
docker build -t hello-devops:v1 .
```

Run it:

```
docker run --rm hello-devops:v1
```

Output:

```
Hello, DevOps!
```

The image uses the **v1** tag instead of **latest**. This was required for the local Nomad setup because the Docker driver attempted to pull the **latest** image from a registry.

```
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assesment$ docker build -t hello-devops:v1 .
[+] Building 2.1s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 127B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/python:3.12-slim@sha256:2c941e860699f878900b0edc2403613c234d4b32eda3cc9fa7036991a2a63c4a
=> => resolve docker.io/library/python:3.12-slim@sha256:2c941e860699f878900b0edc2403613c234d4b32eda3cc9fa7036991a2a63c4a
=> [internal] load build context
=> => transferring context: 29B
=> CACHED [2/3] WORKDIR /app
=> CACHED [3/3] COPY hello.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:a343f79d8252fde8d00022c9e11964dcfacb14a4f0269227ff4b1df741160b31
=> => exporting config sha256:810a7e36eafdadd1d866d34376b4937684034b68f154205bcfb7ec87e0b2313
=> => exporting attestation manifest sha256:a8a54877780858a796f48a82fdd6349d1b2bce6d4939273774040e6dac6171a9
=> => exporting manifest list sha256:b37b17a19586c8e40af765dfca10e08492f6b17522c77eebce1cf8604a29cd0d
=> => naming to docker.io/library/hello-devops:v1
=> => unpacking to docker.io/library/hello-devops:v1
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assesment$ docker run --rm hello-devops:v1
Hello, DevOps!
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assesment$ |
```

## 4. CI/CD with GitHub Actions

The workflow is located at:

```
.github/workflows/ci.yml
```

It runs on pushes and pull requests to **main**.

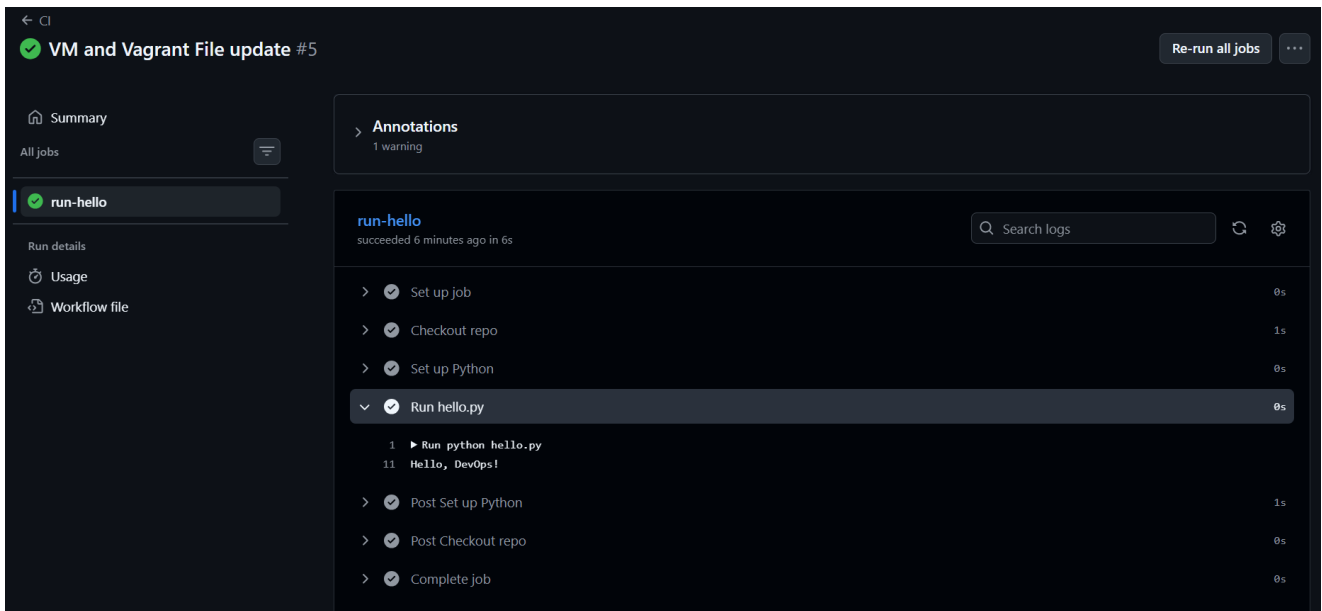
The workflow:

1. Checks out the repository.
2. Sets up Python 3.12.
3. Runs **python hello.py**.
4. Runs **scripts/sysinfo.sh**.

The workflow status can be checked from the **Actions** tab on GitHub.

The screenshot shows the GitHub Actions interface for the repository 'Springer-Capital-DevOps-x-Acumen-Final-Assessment'. The 'Actions' tab is selected, displaying a list of workflow runs. The left sidebar shows navigation options like 'All workflows', 'CI', 'Management', 'Caches', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'. The main area shows 'All workflows' with a search bar and a table of recent runs.

| Workflow                                                      | Event                                           | Status | Branch | Actor | Time                     |
|---------------------------------------------------------------|-------------------------------------------------|--------|--------|-------|--------------------------|
| Help us improve GitHub Actions                                |                                                 |        |        |       |                          |
| VM and Vagrant File update                                    | CI #5: Commit a6755ee pushed by Yajat-prabhakar | main   |        |       | 6 minutes ago            |
| VM and MLFlow(for extra credit)                               | CI #4: Commit 9a8fa27 pushed by Yajat-prabhakar | main   |        |       | 36 minutes ago           |
| Complete Loki monitoring step with working Docker logging ... | CI #3: Commit fd0bb43 pushed by Yajat-prabhakar | main   |        |       | Today at 10:21 AM        |
| Update Nomad job, README, and Docker tag                      | CI #2: Commit 08692c8 pushed by Yajat-prabhakar | main   |        |       | Today at 9:58 AM         |
| CI FIX                                                        | CI #1: Commit 2446318 pushed by Yajat-prabhakar | main   |        |       | Aug 18, 9:34 PM GMT+5:30 |



## 5. Job Deployment with Nomad

The Nomad job is defined in:

```
nomad/hello.nomad
```

It runs the Docker image created in the previous step.

### Environment

The project was initially run on Windows. The native Windows Nomad agent could not communicate with Docker Desktop when Docker Desktop was using Linux containers. The Docker driver reported as unhealthy because it was looking for the Windows-container `npipe` endpoint.

The setup was moved to WSL2 with Docker Desktop's WSL integration. Nomad could then access Docker through:

```
unix:///var/run/docker.sock
```

No additional Docker driver configuration was needed.

Job Type: `batch`

The assessment brief suggests:

```
type = "service"
```

However, `hello.py` runs once and exits. A service job expects a long-running process, so Nomad restarted the task after each successful exit and eventually marked the allocation as failed after the restart attempts were exhausted.

The job therefore uses:

```
type = "batch"
```

A successful run exits with code `0` and the allocation is reported as `complete`.

## Image Tag

The first version of the job used:

```
hello-devops:latest
```

Nomad attempted to pull that image from a registry and returned:

```
pull access denied ... repository does not exist
```

The image was changed to:

```
hello-devops:v1
```

with:

```
force_pull = false
```

This allowed Nomad to use the locally built image.

## Final Job File

```
job "hello" {
  datacenters = ["dc1"]
  type        = "batch"

  group "hello-group" {
    count = 1

    task "hello-task" {
```

```
driver = "docker"

config {
  image      = "hello-devops:v1"
  force_pull = false
}

resources {
  cpu    = 100
  memory = 128
}

restart {
  attempts = 0
  mode     = "fail"
}
}
```

## Prerequisites

Start a Nomad development agent:

```
nomad agent -dev
```

When running on Windows, the agent should be started from WSL2 so that it can access Docker Desktop.

The Docker image must also exist locally:

```
docker build -t hello-devops:v1 .
```

## Run the Job

```
nomad job run nomad/hello.nomad
nomad job status hello
```

Get the allocation ID from the job status:

```
nomad alloc status <alloc-id>
nomad alloc logs <alloc-id>
```

## Actual Output

```
Client Status      = complete
Client Description  = All tasks have completed
...
Recent Events:
Time              Type           Description
2026-08-19T06:02:19Z Terminated Exit Code: 0
2026-08-19T06:02:19Z Started      Task started by client
2026-08-19T06:02:18Z Task Setup   Building Task Directory
2026-08-19T06:02:18Z Received     Task received by client
```

```
$ nomad alloc logs ab069b41
Hello, DevOps!
```

```
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assesment$ nomad alloc status ab069b41
ID                = ab069b41-8a5f-9fc3-89e7-1a75dae25735
Eval ID           = 090db7f4
Name              = hello.hello-group[0]
Node ID           = b5562a26
Node Name         = LAPTOP-GTPP02FF
Job ID            = hello
Job Version       = 0
Client Status     = complete
Client Description = All tasks have completed
Desired Status    = run
Desired Description = <none>
Created           = 51s ago
Modified          = 50s ago

Task "hello-task" is "dead"
Task Resources:
CPU      Memory  Disk    Addresses
100 MHz  128 MiB   300 MiB

Task Events:
Started At    = 2026-08-19T06:02:19Z
Finished At   = 2026-08-19T06:02:19Z
Total Restarts = 0
Last Restart  = N/A

Recent Events:
Time              Type           Description
2026-08-19T06:02:19Z Terminated Exit Code: 0
2026-08-19T06:02:19Z Started      Task started by client
2026-08-19T06:02:18Z Task Setup   Building Task Directory
2026-08-19T06:02:18Z Received     Task received by client

==> View allocation details in the Web UI: http://127.0.0.1:4646/ui/allocations/ab069b41-8a5f-9fc3-89e7-1a75dae25735
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assesment$ nomad alloc logs ab069b41
Hello, DevOps!
```

## 6. Monitoring with Grafana Loki

Loki was run locally in Docker.

The first log collection attempt used Promtail with Docker auto-discovery. It failed because of a Docker API version mismatch:

```
client version 1.42 is too old
```

The working setup uses Docker's native Loki logging driver. Container stdout/stderr is sent directly to Loki.

The detailed setup is in:

```
monitoring/loki_setup.txt
```

The Promtail configuration used during the initial attempt is kept in:

```
monitoring/promtail-config.yaml
```

## Working Setup

Start Loki:

```
docker run -d --name=loki -p 3100:3100 grafana/loki:2.9.0 \
  -config.file=/etc/loki/local-config.yaml
```

Install the Docker logging driver:

```
docker plugin install grafana/loki-docker-driver:latest \
  --alias loki --grant-all-permissions
```

Run the container with the Loki logging driver:

```
docker run --rm --name hello-run \
  --log-driver=loki \
  --log-opt loki-url="http://host.docker.internal:3100/loki/api/v1/push" \
  hello-devops:v1
```

## Query the Logs

```
curl -G -s "http://localhost:3100/loki/api/v1/query_range" \
  --data-urlencode 'query={container_name="hello-run"}' \
  --data-urlencode 'limit=20' | python3 -m json.tool
```

## Actual Result

```
"stream": {
  "container_name": "hello-run",
  "filename":
"/var/log/docker/8e1940e67ea57d010f807fb5715069ad6ad3363726c0fa12655198aab5cb12
7d/json.log",
  "host": "docker-desktop",
  "source": "stdout"
},
"values": [
  ["1787121271009687203", "Hello, DevOps!"]
]
```

 Loki query result

---

## 7. Extra Credit

### MLflow Tracking

mlflow/log\_experiment.py logs:

- learning\_rate
- epochs
- Accuracy for each epoch
- Loss for each epoch
- hello.py as an artifact

MLflow data is stored under:

```
~/mlflow-runs/devops-intern-final
```

The storage is kept on the native Linux filesystem instead of `/mnt/c/...`. This avoids the file-copying permission issue encountered when MLflow was run from a Windows-mounted WSL2 path.

Run it with:

```
cd mlflow
sudo apt install -y python3-pip
pip3 install -r requirements.txt --break-system-packages
python3 log_experiment.py
```

#### Actual output:

```
epoch=1 accuracy=0.561 loss=0.8564
epoch=2 accuracy=0.6577 loss=0.694
```

```
epoch=3 accuracy=0.7553 loss=0.5627
epoch=4 accuracy=0.8262 loss=0.398
epoch=5 accuracy=0.8891 loss=0.2358
MLflow run complete.
```

```
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assesment/mlflow$ python3 log_experiment.py
epoch=1 accuracy=0.561 loss=0.8564
epoch=2 accuracy=0.6577 loss=0.694
epoch=3 accuracy=0.7553 loss=0.5627
epoch=4 accuracy=0.8262 loss=0.398
epoch=5 accuracy=0.8891 loss=0.2358
MLflow run complete.
yajat@LAPTOP-GTPP02FF:/mnt/c/Users/Yajat/Springer Capital DevOps x Acumen Final Assesment/mlflow$ |
```

View the run:

```
mlflow ui --backend-store-uri sqlite:/// $HOME/mlflow-runs/devops-intern-
final/mlflow.db
```

Then open:

```
http://localhost:5000
```

and select:

```
devops-intern-final-demo
```

## VM Deployment (VirtualBox)

A VirtualBox VM was created with Vagrant to run the Docker/Nomad job separately from WSL2.

The Vagrant configuration is in:

```
vm/Vagrantfile
```

The VM setup notes and troubleshooting details are in:

```
vm/vm_setup_notes.txt
```

One issue encountered during a rebuild was a VirtualBox/Hyper-V virtualization conflict that caused boot/SSH timeouts. Enabling VirtualBox's Hyper-V paravirtualization interface resolved the issue.

### Actual output from the VM:

#### Task Events:

| Time                 | Type       | Description             |
|----------------------|------------|-------------------------|
| 2026-08-19T07:30:14Z | Terminated | Exit Code: 0            |
| 2026-08-19T07:30:14Z | Started    | Task started by client  |
| 2026-08-19T07:30:12Z | Task Setup | Building Task Directory |
| 2026-08-19T07:30:12Z | Received   | Task received by client |

```
$ nomad alloc logs 9600fd36
```

```
Hello, DevOps!
```

```
vagrant@ubuntu-jammy:~$ cd Springer-Capital-DevOps-x-Acumen-Final-Assessment/
vagrant@ubuntu-jammy:~/Springer-Capital-DevOps-x-Acumen-Final-Assessment$ nomad
job run nomad/hello.nomad
nomad job status hello

=> View this job in the Web UI: http://127.0.0.1:4646/ui/jobs/hello@default

==> 2026-08-19T07:30:12Z: Monitoring evaluation "1fc0fd46"
2026-08-19T07:30:12Z: Evaluation triggered by job "hello"
2026-08-19T07:30:12Z: Allocation "9600fd36" created: node "31bb5b88", group
"hello-group"
2026-08-19T07:30:13Z: Evaluation status changed: "pending" -> "complete"
==> 2026-08-19T07:30:13Z: Evaluation "1fc0fd46" finished with status "complete"
ID           = hello
Name          = hello
Submit Date   = 2026-08-19T07:30:12Z
Type          = batch
Priority       = 50
Datacenters   = dc1
Namespace     = default
Node Pool     = default
Task "hello-task" is "dead"
Task Resources:
CPU    Memory    Disk    Addresses
100 MHz 128 MiB 300 MiB

Task Events:
Started At   = 2026-08-19T07:30:14Z
Finished At  = 2026-08-19T07:30:14Z
Total Restarts = 0
Last Restart  = N/A

Recent Events:
Time          Type          Description
2026-08-19T07:30:14Z Terminated  Exit Code: 0
2026-08-19T07:30:14Z Started      Task started by client
2026-08-19T07:30:12Z Task Setup   Building Task Directory
2026-08-19T07:30:12Z Received     Task received by client

=> View allocation details in the Web UI: http://127.0.0.1:4646/ui/allocations/9600fd36-4d40-f0a3-e950-54a545b5c181
vagrant@ubuntu-jammy:~/Springer-Capital-DevOps-x-Acumen-Final-Assessment$ nomad alloc logs 9600fd36
Hello, DevOps!
vagrant@ubuntu-jammy:~/Springer-Capital-DevOps-x-Acumen-Final-Assessment$ |
```

## Repository Structure

```
.
├── README.md
├── hello.py
├── Dockerfile
├── scripts/
│   └── sysinfo.sh
├── .github/
│   └── workflows/
│       └── ci.yml
├── nomad/
│   └── hello.nomad
├── monitoring/
└── loki_setup.txt
```

```
|   └─ promtail-config.yaml
├─ mlflow/
|   └─ log_experiment.py
|   └─ requirements.txt
├─ vm/
|   └─ Vagrantfile
|   └─ vm_setup_notes.txt
└─ docs/
    └─ 01-github-repo.png
    └─ 02-sysinfo.png
    └─ 03-docker-run.png
    └─ 04-ci-actions-list.png
    └─ 04b-ci-run-detail.png
    └─ 05-nomad-run.png
    └─ 06-mlflow-run.png
    └─ 07-loki-query.png
    └─ 08-vm-nomad-run.png
```